Fuzzing Guiado por Contrato para APIs REST

Title: Contract-Guided Fuzzing for REST APIs

Levi da Rosa Gomes (aluno)[1], **Fábio Austrian Zanin (orientador)**[1]

[1] Universidade Regional Integrada do Alto Uruguai e das Missões
Departamento de Engenharias e Ciência da Computação
Caixa Postal 743 – 99.709-910 – Erechim – RS – Brasil

103495@uricer.edu.br, faz@uricer.edu.br

Abstract. *Divergences between REST API implementations and their OpenAPI contracts can affect integration reliability and reveal robustness, conformance, and access-control anomalies. This work presents a black-box, contract-guided fuzzer and investigates the use of a fuzzy inference system to prioritize the reported anomalies. The tool parses a supported subset of OpenAPI, generates baseline and mutated inputs, executes HTTP requests, detects contractual divergences, and preserves reproducible artifacts. Four APIs supported the development and audit of the prioritization model; after its revision was frozen, Meilisearch was used as an external holdout. The development pilot executed 958 requests and produced 161 behavioral families. The initial model reproduced the same categorical decisions as a status-only baseline, with accuracy 0.298. On 122 supported holdout families, the revised model agreed with a blind operational reference in 120 cases, compared with 4 for the baseline. These partial results concern technical priority, not actual security impact; the final experimental protocol remains subject to advisor review.*
Keywords *REST APIs, OpenAPI, Fuzzing, Contract Testing, Fuzzy Logic.*

Resumo. *Divergências entre implementações de APIs REST e seus contratos OpenAPI podem afetar a confiabilidade das integrações e revelar anomalias de robustez, conformidade e controle de acesso. Este trabalho apresenta um fuzzer de caixa-preta guiado por contrato e investiga o uso de um sistema de inferência fuzzy para priorizar as anomalias reportadas. A ferramenta interpreta um subconjunto de OpenAPI, gera entradas-base e mutações, executa requisições HTTP, detecta divergências contratuais e preserva artefatos reproduzíveis. Quatro APIs apoiaram o desenvolvimento e a auditoria do modelo de priorização; após o congelamento de sua revisão, o Meilisearch foi utilizado como holdout externo. O piloto de desenvolvimento executou 958 requisições e produziu 161 famílias comportamentais. O modelo inicial reproduziu as mesmas decisões categóricas de um baseline baseado apenas em status, com acurácia 0,298. Nas 122 famílias sustentadas do holdout, o modelo revisado concordou com uma referência operacional cega em 120 casos, contra 4 do baseline. Esses resultados parciais tratam de prioridade técnica, não de impacto real de segurança; o protocolo experimental final permanece sujeito à avaliação do orientador.*
Palavras-Chave *APIs REST, OpenAPI, Fuzzing, Teste de Contrato, Lógica Fuzzy.*

1. Introdução

APIs REST tornaram-se parte essencial da infraestrutura de sistemas modernos, conectando serviços, aplicações web e dispositivos móveis [Arcuri et al. 2026]. Nesse cenário, falhas de robustez e inconsistências entre o comportamento da API e o contrato publicado afetam a confiabilidade das integrações e o custo de manutenção. Testes funcionais tradicionais

validam cenários esperados, mas podem deixar de exercitar valores limite, formatos inválidos e combinações pouco usuais de parâmetros.

Contratos como a especificação OpenAPI descrevem operações, estruturas, tipos, restrições e respostas esperadas. Eles podem orientar a geração sistemática de entradas válidas e inválidas e servir como oráculo parcial para identificar divergências entre especificação e implementação. Essa abordagem permite observar a API externamente, sem instrumentação de código, em um contexto *black-box* relacionado à noção de contrato de software [Meyer 1992].

Este trabalho apresenta uma ferramenta de *fuzzing* automatizado para APIs REST, implementada em Python e orientada por um subconjunto de OpenAPI. O pipeline interpreta o contrato, gera valores-base e mutações, materializa um plano de execução, envia requisições, confronta as respostas com o contrato e produz artefatos rastreáveis. Um sistema de inferência fuzzy Mamdani atribui prioridade técnica às anomalias reportadas depois da classificação contratual.

A questão central provisória é: em que medida um *fuzzer* de caixa-preta, guiado por um subconjunto de OpenAPI, produz achados tecnicamente relevantes em APIs REST e em que medida um sistema de inferência fuzzy fornece uma ordenação de prioridade coerente com critérios definidos independentemente de seu escore?

A avaliação é organizada em três dimensões complementares: (i) capacidade de geração e detecção de diferentes anomalias contratuais e comportamentais; (ii) coerência da prioridade técnica atribuída aos achados; e (iii) estabilidade dos comportamentos observados quando há campanhas repetidas. A terceira dimensão foi examinada no *holdout*; sua extensão ao restante do corpus depende da definição do protocolo final com o orientador.

Quatro APIs foram utilizadas como pilotos de desenvolvimento, permitindo auditar a configuração inicial e formular um modelo revisado. Após o congelamento das regras, uma quinta API foi empregada como *holdout* externo. A prioridade foi comparada com uma referência operacional cega, construída sem revelar o escore e assistida por agentes independentes. Esses resultados são evidência parcial e base para discutir o delineamento definitivo; não implicam superioridade geral sobre outros *fuzzers* nem estimativa de impacto real de segurança.

O escopo limita-se a APIs REST descritas em OpenAPI. Não são abrangidos sequenciamento stateful geral, fluxos completos de OAuth2, avaliação de carga, cobertura de código ou confirmação automática de vulnerabilidades. Um achado pode representar *contract drift*, incompatibilidade funcional, robustez ou comportamento de controle de acesso e ainda exigir interpretação antes de ser tratado como defeito.

O restante deste artigo está organizado da seguinte forma. A Seção 2 apresenta os conceitos necessários. A Seção 3 posiciona a proposta diante das ferramentas relacionadas. A Seção 4 descreve o desenho da avaliação, e a Seção 5 apresenta a ferramenta. A Seção 6 registra os resultados parciais. Por fim, a Seção 7 sintetiza o checkpoint e suas limitações.

2. Fundamentação Teórica

Esta seção delimita os conceitos necessários para compreender a geração de testes guiada por contratos e o mecanismo de priorização. A versão definitiva incorporará as fontes primárias já mapeadas na revisão bibliográfica.

2.1. APIs REST e Especificação OpenAPI

Neste trabalho, o documento OpenAPI exerce dois papéis. Primeiro, fornece estrutura para gerar operações, parâmetros e corpos de requisição. Segundo, atua como oráculo parcial ao descrever

status e schemas esperados nas respostas. Esse oráculo é incompleto: uma API pode aplicar regras de negócio ausentes do contrato, e o contrato também pode divergir da implementação sem que exista falha de segurança.

[Sketch para orientação: incorporar definições e fontes primárias sobre REST, OpenAPI e contrato como oráculo parcial.]

2.2. Teste Negativo e Fuzzing

Teste negativo exercita entradas inválidas, limites e combinações adversas para observar se o sistema rejeita dados de forma controlada. O *fuzzing* pode ser geracional, quando produz casos a partir de um modelo, ou mutacional, quando transforma entradas existentes. A ferramenta combina as duas ideias: gera um valor-base do contrato e aplica operadores tipados. Diferentemente de fuzzers guiados por cobertura, a observação permanece *black-box*.

[Sketch para orientação: incorporar fontes primárias sobre teste negativo, robustez e fuzzing geracional/mutacional.]

2.3. Fuzzing de APIs REST Guiado por Contrato

APIs introduzem desafios de dados estruturados, autenticação, dependências entre recursos, estado e serialização. OpenAPI permite orientar entradas e comparar respostas, mas não resolve automaticamente essas dependências nem garante a completude do oráculo. Por isso, rejeições seguras não documentadas, aceitações indevidas, erros de servidor e divergências de schema precisam ser descritos separadamente.

[Sketch para orientação: consolidar aqui as fontes de fuzzing de APIs e os limites conhecidos de abordagens exclusivamente black-box.]

2.4. Lógica Fuzzy e Prioridade Técnica

Um sistema Mamdani representa conceitos graduais por funções de pertinência, combina antecedentes em regras linguísticas, agrega suas ativações e produz um valor contínuo por desfuzzificação. Essa estrutura favorece uma priorização interpretável, pois permite preservar entradas, pertinências e regras ativadas.

Neste trabalho, a saída representa prioridade técnica da anomalia. Ela não confirma vulnerabilidade, explorabilidade, impacto de negócio ou urgência real de correção. O modelo revisado utiliza contradição comportamental, força da evidência e exposição de controle de acesso como antecedentes. A detecção contratual ocorre antes da priorização e constitui uma responsabilidade conceitualmente distinta.

[Sketch para orientação: incorporar fundamentos de conjuntos fuzzy, inferência Mamdani e priorização, distinguindo parâmetros heurísticos de parâmetros apoiados pela literatura.]

3. Trabalhos Relacionados

O uso de OpenAPI para geração automática de testes está estabelecido por ferramentas com estratégias e objetivos distintos. A comparação não busca uma ferramenta universalmente superior, mas posiciona a proposta quanto à geração de casos negativos, exploração de estado, oráculos, reprodução e priorização.

[Sketch para orientação: as versões consultadas e as fontes primárias de cada ferramenta ainda serão incorporadas à bibliografia. A matriz expressa um posicionamento provisório, não uma comparação experimental concluída.]

Entre as ferramentas levantadas, Schemathesis permanece como o candidato mais próximo para uma comparação experimental limitada. Sua inclusão ainda não foi definida:

Tabela 1. Matriz preliminar de posicionamento das ferramentas relacionadas

Ferramenta	Abordagem principal	Estado	Papel no TCC
Schemathesis	Testes baseados em propriedades a partir de OpenAPI	Parcialmente stateful	Candidato mais próximo a uma comparação experimental; decisão pendente de orientação
RESTler	Gramática e sequências derivadas do contrato	Stateful	Referência para dependências, deduplicação e reprodução
RESTest	Geração black-box aleatória e baseada em restrições	Suporte condicionado	Referência para casos nominais, inválidos e oráculos
RestTestGen	Framework extensível de teste black-box	Extensível	Comparação arquitetural com parser, mutadores e oráculos
EvoMaster	Geração orientada por busca	Black-box e white-box	Contraste com técnicas search-based e feedback interno
Dredd	Execução de exemplos e teste de conformidade	Hooks manuais	Antecedente de teste por contrato, não baseline de fuzzing

comparabilidade, custo e contribuição para a questão central serão discutidos com o orientador. As demais ferramentas cumprem papel de referência conceitual ou arquitetural, sem pretensão de equivalência experimental direta.

No corpus bibliográfico levantado, não foi identificado um mecanismo diretamente equivalente à integração proposta entre sinais contratuais, evidência observada e um escore fuzzy auditável. Essa observação não constitui alegação de inexistência. A contribuição investigada é a integração e a avaliação desse mecanismo no pipeline desenvolvido.

4. Metodologia

Esta pesquisa é aplicada e experimental e avalia um *fuzzer black-box* orientado por contratos OpenAPI. A investigação considera três dimensões complementares: capacidade de caracterizar anomalias técnicas em APIs distintas, coerência da priorização fuzzy e estabilidade dos comportamentos observados em campanhas repetidas. O construto do FIS é a prioridade técnica da anomalia; seu escore não representa explorabilidade, impacto real nem gravidade de segurança demonstrada.

4.1. Desenho do Estudo e Papel das Aplicações

VAmPI, Spring Petclinic REST, crAPI e RealWorld constituíram o corpus de desenvolvimento e caracterização. Os resultados dessas aplicações permitiram auditar a configuração inicial do mecanismo de priorização e formular sua revisão. O Meilisearch v1.45.1 foi utilizado posteriormente como unidade confirmatória externa, sem participar do ajuste das novas regras.

As aplicações foram executadas localmente em versões fixadas e reiniciáveis. Contratos, imagens, configuração do fuzzer e procedimentos de inicialização foram registrados para separar mudanças do objeto avaliado de variações do ambiente. O Meilisearch foi submetido a cinco campanhas completas, com reset entre elas. O modelo de coleta adotado no fechamento do TCC será definido com o orientador; este artigo-sketch não pressupõe novas campanhas.

4.2. Geração dos Testes e Níveis de Mutação

A geração começa com um caso compatível com as informações interpretadas do contrato. A partir dele, o motor aplica operadores tipados sobre parâmetros e corpos. Três níveis organizam a exploração:

- **Nível 0 (L0) — baseline:** executa a entrada-base gerada para satisfazer o contrato e estabelece uma referência nominal para a operação;

- **Nível 1 (L1) — mutação isolada:** aplica uma alteração por caso, permitindo relacionar a resposta com um operador específico;
- **Nível 2 (L2) — mutações combinadas:** combina duas alterações compatíveis para explorar interações, preservando orçamento próprio e os casos isolados de L1.

Essa progressão distingue comportamento nominal, efeito individual e interação entre mutações. O plano também inclui, quando habilitados, testes de operações protegidas sem credencial ou com credencial inválida. Limites de casos, concorrência, atraso e timeout são registrados como parte da configuração.

4.3. Protocolo Experimental e Estratégia de Avaliação

Cada campanha segue o fluxo contrato, plano, execução, análise e pós-processamento. Um manifesto registra identificador do experimento, API, repetição, revisão do código, estado Git, hashes, ambiente, configuração, horários e contagens. O plano e os resultados são correlacionados por `case_id`; campanhas incompletas ou inválidas devem permanecer identificáveis, em vez de desaparecer das evidências.

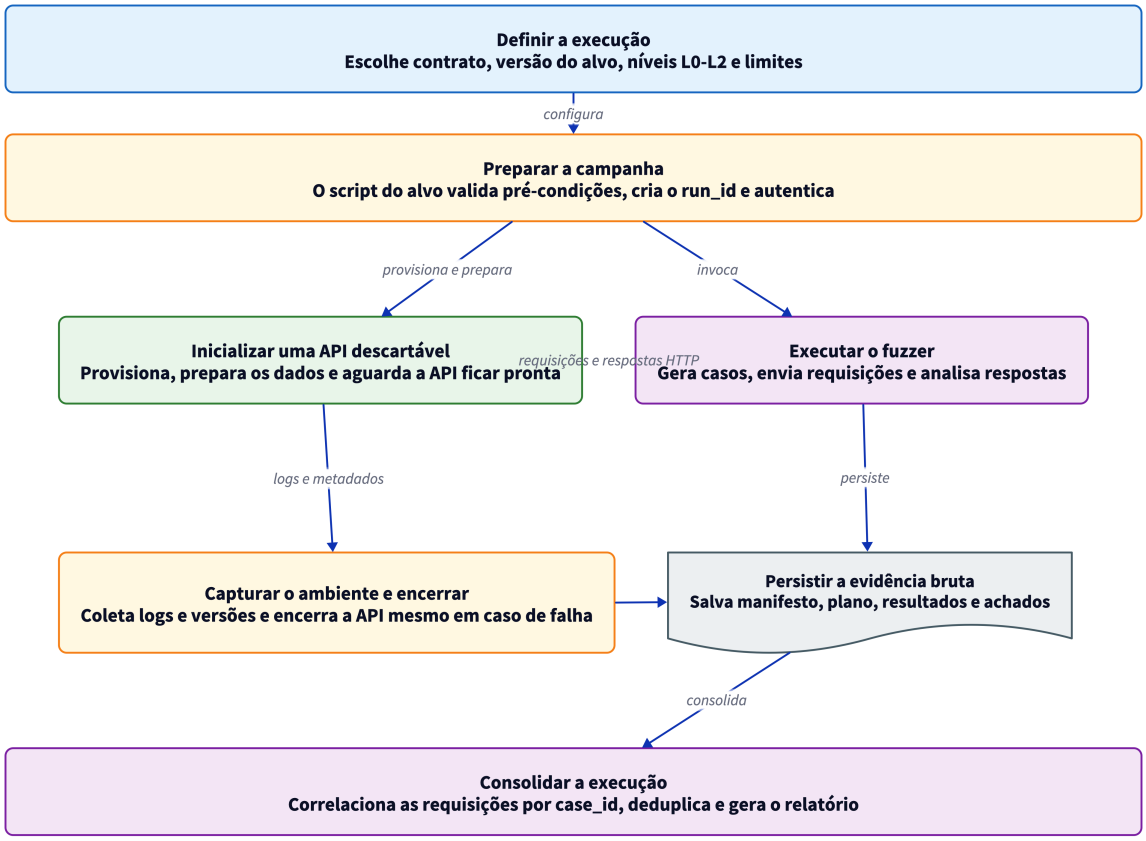


Figura 1. Ciclo de uma execução experimental isolada. O script específico prepara o alvo descartável e invoca o fuzzer; a evidência da execução é completada com a proveniência do alvo antes da deduplicação e consolidação.

Cinco níveis de observação são mantidos separados: requisição, achado bruto, assinatura estrita, campanha e família comportamental. A assinatura estrita agrupa observações com a mesma aplicação, operação, classificação, status e resposta normalizada. Famílias comportamentais reúnem assinaturas por endpoint, tipo de divergência e contexto factual para avaliar prioridade. Esses grupos são unidades operacionais e não alegações de causa raiz.

A referência de prioridade foi construída sem usar o próprio score do FIS como resposta. As fichas apresentaram contrato, contexto da requisição e resposta, mas ocultaram classificação do analisador, faixa, antecedentes, pertinências e regras fuzzy. Três agentes em contextos isolados aplicaram uma rubrica explícita; casos sem maioria foram encaminhados a um quarto agente, também cego ao FIS e aos votos anteriores. O procedimento produz uma referência operacional independente do score, não um *gold standard* humano nem confirmação de vulnerabilidade.

O modelo é comparado com um baseline baseado somente no desvio de status HTTP. A análise utiliza matriz de confusão, acurácia, macro-F1, erro ordinal médio e correlação de Spearman, limitando métricas às classes efetivamente observadas. A estabilidade é examinada pela reobservação de assinaturas entre campanhas, sem protocolo separado de replay imediato.

As principais ameaças à validade são o OpenAPI como oráculo parcial, suporte incompleto à especificação, deduplicação black-box, quantidade de aplicações, desbalanceamento das classes e possível correlação entre julgadores agentes. O corpus de desenvolvimento não demonstra generalização do modelo revisado, e o holdout não contém referência HIGH. As conclusões devem permanecer limitadas às aplicações, contratos, versões e configurações avaliados.

5. Desenvolvimento da Ferramenta

O OpenAPI Contract Fuzzer foi implementado em Python como um pipeline modular. A organização separa interpretação do contrato, geração dos casos, execução HTTP, análise das respostas, priorização e persistência dos artefatos. Esta seção descreve as responsabilidades necessárias para compreender e reproduzir o fluxo, sem transformar a estrutura de arquivos do código em arquitetura do artigo.

5.1. Do Contrato ao Plano de Execução

O parser aceita documentos OpenAPI em YAML ou JSON e os converte em um modelo interno de operações, parâmetros, corpos, respostas e requisitos de segurança. O subconjunto implementado reconhece GET, POST, PUT, PATCH e DELETE; parâmetros de caminho, consulta e cabeçalho; corpos e respostas JSON; referências locais; e composição limitada por `allOf`. Restrições como obrigatoriedade, `readOnly`, enumeração, formato, padrão, tamanhos e limites numéricos são preservadas quando suportadas.

O gerador produz a entrada-base e o motor de mutações percorre sua estrutura para aplicar alterações tipadas. O construtor do plano resolve caminhos, payloads, cabeçalhos, autenticação e estratégia aplicada. Cada entrada recebe um `case_id` e registra se representa baseline, mutação ou teste de autenticação, além dos metadados das mutações. O plano completo é persistido antes das chamadas à API.

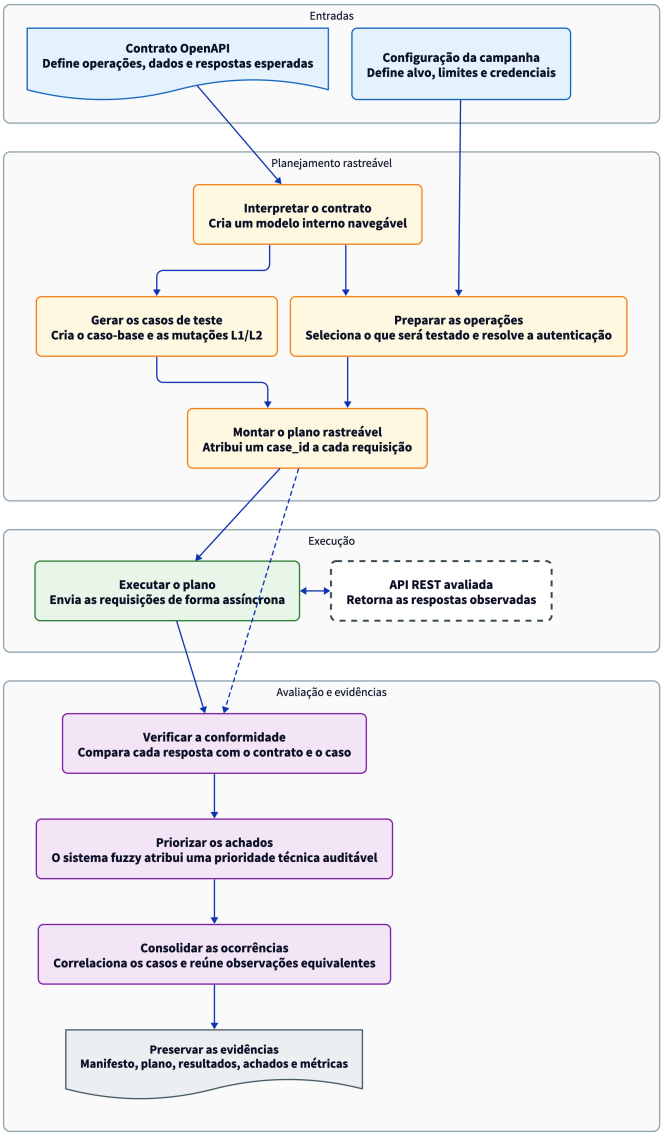


Figura 2. Arquitetura interna do OpenAPI Contract Fuzzer. Contrato e parâmetros são transformados em um plano rastreável; as respostas da API são confrontadas com o comportamento documentado, priorizadas pelo sistema fuzzy e consolidadas em artefatos auditáveis.

5.2. Execução, Análise e Rastreabilidade

A execução utiliza cliente HTTP assíncrono, semáforo de concorrência, atraso e timeout configuráveis. Para cada caso são registrados URL, status, tempo, tamanho, corpo e erro de transporte. Os runners experimentais cuidam da inicialização, do reset e da proveniência específica das aplicações sem alterar o núcleo da ferramenta.

O analisador confronta status e schemas observados com o contrato. Entre as classificações estão falhas de transporte, erros 5xx, status não documentados, incompatibilidades de schema e aceitação de entradas que deveriam ser rejeitadas. Uma rejeição 4xx documentada e compatível é preservada como comportamento esperado e separada dos candidatos. Uma divergência contratual continua relevante mesmo quando a API rejeita a entrada com segurança, pois pode afetar consumidores; por outro lado, uma observação reportada não equivale automaticamente a causa raiz, defeito confirmado ou vulnerabilidade.

O pós-processamento correlaciona plano, resultado e achado por `case_id`, constrói

assinaturas, seleciona representantes e calcula métricas. Cada diretório de campanha reúne `run_manifest.json`, plano de execução, resultados brutos, achados estruturados, conjunto deduplicado e relatórios em Markdown. O manifesto preserva configuração, revisão e estado do código, hashes do contrato e da árvore-fonte, ambiente, horários, contagens e estado da campanha. Cabeçalhos e valores sensíveis são redigidos antes da persistência pública.

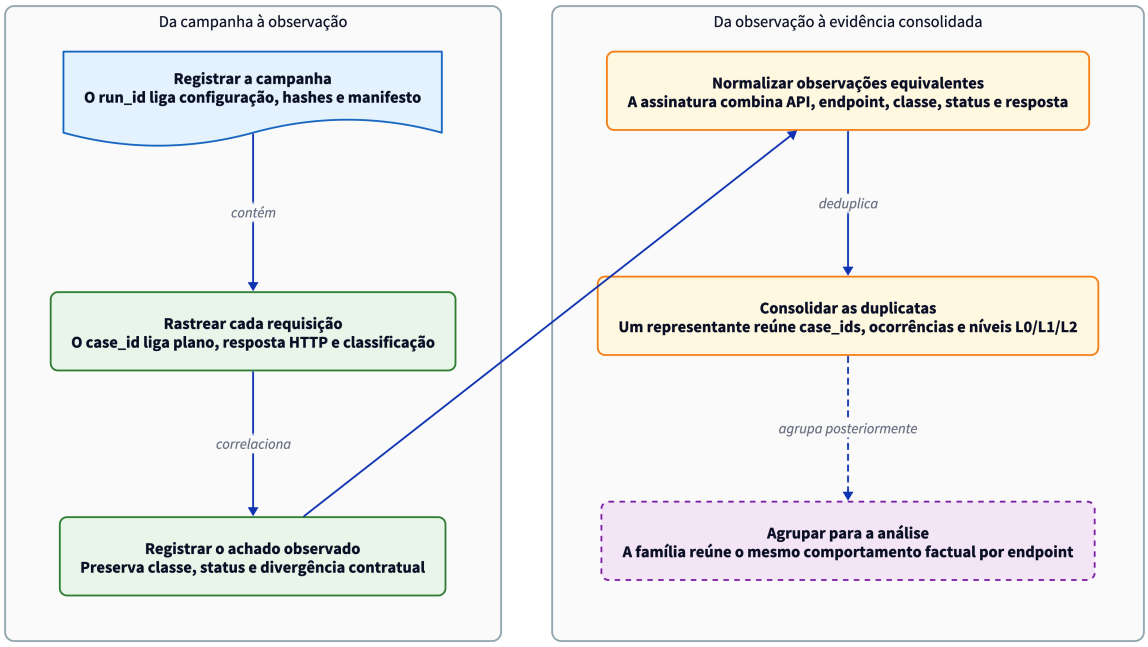


Figura 3. Níveis de observação e rastreabilidade dos achados. O `case_id` relaciona plano, resposta e classificação; a deduplicação produz achados operacionais únicos, e o agrupamento em famílias ocorre em etapa analítica posterior.

Esse conjunto permite rastrear os números do artigo até casos concretos e distinguir falha da aplicação, falha ambiental e campanha incompleta. Uma amostra pública gerada pelo pipeline contém uma execução L2 de cada alvo e demonstra o formato dos artefatos, mas não define o modelo de coleta final.

5.3. Priorização Fuzzy

Depois da classificação contratual, um sistema Mamdani combina sinais observáveis para produzir um escore contínuo e as faixas LOW, MEDIUM e HIGH. O FIS não decide se existe divergência; ele organiza a inspeção dos achados já reportados. Entradas, pertinências, regras ativadas e eventual fallback são preservados nos artefatos para tornar a decisão auditável.

O construto é prioridade técnica da anomalia, abrangendo contract drift, compatibilidade funcional, robustez e controle de acesso. O escore não estima impacto real, explorabilidade nem gravidade de segurança. A evolução do modelo e os sinais utilizados em sua revisão são apresentados uma única vez na Seção 6.2, junto das evidências que motivaram a mudança.

A cobertura de OpenAPI permanece parcial. Outros media types, `oneOf`, `anyOf`, referências externas e parte das regras de serialização não são suportados. A ferramenta também não resolve dependências gerais entre recursos ou sequências stateful; identificadores dinâmicos podem apontar para recursos inexistentes. Esses limites restringem as operações exercitáveis e acompanham a interpretação dos resultados.

6. Resultados e Discussão

Os resultados disponíveis descrevem uma trajetória experimental: os pilotos caracterizaram o funcionamento do pipeline e expuseram limitações do mecanismo de priorização; a auditoria orientou uma revisão; e o modelo congelado foi aplicado posteriormente a uma API que não participou de seu desenvolvimento. Essa sequência permite apresentar evidências parciais sem confundir corpus de desenvolvimento, confirmação externa e uma futura coleta final.

6.1. Caracterização dos Pilotos e dos Níveis de Mutação

As campanhas de Nível 2 contra VAmPI, Spring Petclinic REST, crAPI e RealWorld executaram 958 de 958 requisições. Desse total, 284 eram casos L2 com duas mutações; os demais preservaram baselines e mutações isoladas. A deduplicação produziu 265 assinaturas estritas, reunidas em 161 famílias comportamentais. A referência cega distribuiu essas famílias em 99 LOW, 59 MEDIUM e 3 HIGH.

O Nível 0 forneceu a observação nominal das operações; o Nível 1 isolou o efeito dos operadores; e o Nível 2 ampliou a variedade de manifestações, especialmente na crAPI. Entretanto, as combinações não produziram uma nova família HIGH exclusiva. O resultado não torna L2 irrelevante: interações entre entradas, contract drift, respostas incompatíveis e variações de robustez também fazem parte do objeto, mesmo quando não indicam vulnerabilidade grave.

6.2. Auditoria e Revisão do Modelo de Priorização

A configuração inicial do FIS combinava desvio de status HTTP, tempo de resposta e tamanho da resposta. A avaliação cega mostrou que o status dominava as decisões: neutralizar tempo ou tamanho não alterou a faixa de nenhuma das 161 famílias. O modelo e o baseline baseado somente em status obtiveram a mesma acurácia, 0,298, o mesmo macro-F1, 0,254, e o mesmo erro ordinal médio, 0,702.

A matriz de confusão revelou 67 famílias LOW elevadas a MEDIUM e 43 famílias MEDIUM elevadas a HIGH. O mecanismo superpriorizava rejeições seguras, 4xx não documentados e falhas 5xx localizadas. Uma rejeição não documentada pode representar contract drift, mas não deve receber automaticamente a mesma prioridade de uma operação válida indisponível, de uma aceitação indevida ou de uma falha recorrente.

Esse diagnóstico levou à revisão do construto e dos antecedentes. O modelo revisado utiliza contradição comportamental, força da evidência e exposição de controle de acesso. A contradição representa a incompatibilidade entre o caso, o status, o schema e o comportamento observado; a evidência distingue manifestações isoladas de determinadas recorrências; e a exposição de acesso captura operações protegidas que aceitam credencial ausente ou inválida. Seis regras Mamdani agregam esses sinais e produzem o escore contínuo.

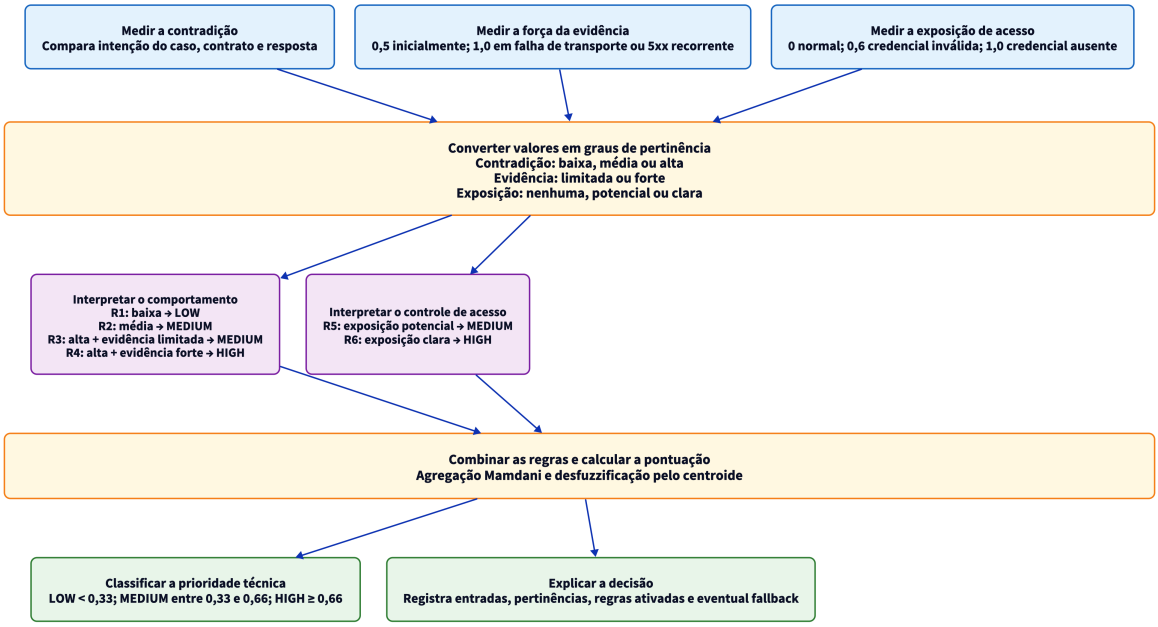


Figura 4. Fluxo auditável do sistema Mamdani de priorização. Sinais do contrato, da resposta e da autenticação são fuzzificados e combinados por seis regras. A saída representa prioridade técnica, não confirmação de vulnerabilidade.

No conjunto de desenvolvimento, o modelo revisado reproduziu as 161 faixas da rubrica, com macro-F1 1,000 e erro ordinal zero. Esse resultado demonstra que a implementação expressa a rubrica formulada, mas não constitui evidência de generalização, pois a mesma amostra foi usada para diagnosticar e revisar o mecanismo. Por isso, regras e parâmetros foram congelados antes da avaliação externa.

6.3. Avaliação Externa no Meilisearch

O modelo congelado foi aplicado ao Meilisearch v1.45.1 em cinco campanhas, cada uma com 85 operações selecionadas e 477 requisições concluídas. As assinaturas estritas apresentaram interseção 216 e união 276. A variação veio dos campos `requestUid` e `processingTimeMs` em `POST /multi-search`. Após normalização restrita a esses dois campos, cada repetição apresentou as mesmas 222 assinaturas semânticas.

As assinaturas foram agrupadas em 134 famílias. A referência final classificou 122 como sustentadas e 12 como indeterminadas. Entre as sustentadas, 116 receberam LOW, 6 receberam MEDIUM e nenhuma recebeu HIGH.

O modelo revisado concordou com a referência em 120 das 122 famílias sustentadas; o baseline baseado somente em status acertou 4. A Tabela 2 apresenta as métricas nas classes observadas.

Como 84 das 122 famílias sustentadas eram controles de credencial inválida, foi realizada uma análise sem esses casos. O modelo revisado permaneceu correto em 36 de 38 famílias, contra 4 de 38 do baseline. Os dois desacordos foram preservados: um baseline de `PATCH /experimental-features` com status 415 não documentado e uma mutação válida de `POST /export` que retornou 200 quando o contrato documentava 202. O FIS não foi reajustado com esses casos.

Tabela 2. Comparação confirmatória nas 122 famílias sustentadas

Métrica	Modelo revisado	Status-only
Acurácia	0,9836	0,0328
Macro-F1 (classes observadas)	0,8957	0,0317
Acurácia balanceada	0,8333	0,3333
Erro ordinal médio	0,0164	0,9672
Spearman	0,9088	-0,6981

6.4. Discussão e Limitações

O principal resultado do corpus de desenvolvimento foi explicar por que o modelo inicial não acrescentava separação categórica ao status. A revisão substituiu sinais fáceis de medir, mas pouco discriminativos, por fatos mais próximos da prioridade técnica. A avaliação externa indica que essa operacionalização permaneceu coerente no Meilisearch sem retroalimentar o modelo congelado.

Os resultados não permitem avaliar a faixa HIGH no holdout, estimar gravidade real de segurança ou extrapolar para APIs em geral. A concordância de prioridade entre os agentes foi baixa na confirmação, e a amostra apresentou forte presença de controles de credencial inválida. A referência deve ser entendida como operacional e auditável, não como verdade universal.

A contribuição observada é um processo rastreável para gerar, detectar, deduplicar e priorizar anomalias técnicas, incluindo contract drift, robustez e controle de acesso. O modelo de coleta final e a comparação com Schemathesis serão discutidos com o orientador.

7. Conclusão

Este artigo-sketch registra o estado atual do projeto, não o fechamento da avaliação. Foi implementado um pipeline que interpreta um subconjunto de OpenAPI, gera baselines e mutações, executa testes black-box, identifica divergências, deduplica observações, atribui prioridade técnica e preserva artefatos rastreáveis por campanha.

Os pilotos mostraram que os níveis de mutação exercitam perspectivas complementares: comportamento nominal, alterações isoladas e interações. A auditoria do mecanismo de priorização revelou que sua configuração inicial era dominada pelo status HTTP. A revisão orientada por fatos comportamentais foi congelada e apresentou concordância substancialmente maior que o baseline no holdout Meilisearch.

A contribuição não é a determinação automática de vulnerabilidades ou impacto real, mas uma forma auditável de priorizar contract drift, incompatibilidades funcionais, problemas de robustez e indícios de controle de acesso. As conclusões são limitadas pelo número de aplicações, pelo OpenAPI como oráculo parcial, pela referência assistida por agentes, pelo desbalanceamento das classes e pela ausência de casos HIGH no holdout.

O modelo de coleta final e a manutenção de uma comparação exploratória com Schemathesis serão definidos com o orientador. Essas decisões podem ampliar a avaliação, mas não alteram a separação entre desenvolvimento e confirmação nem os resultados já preservados.

8. Referências

Arcuri, A., Poth, A., Rrjolli, O., Garrett, P., e Galeotti, J. P. (2026). Fuzzing rest apis in industry: Necessary features and open problems.

Meyer, B. (1992). Applying "design by contract". *IEEE Computer*, 25(10):40–51.